

4 Чисельне розв'язання крайової задачі оптимального керування

4.1 Постановка проблематики та розклад траєкторії польоту

У другому розділі, на основі принципу максимуму Понтрягіна, було доведено, що оптимальне керування для задачі м'якої посадки космічного апарату має релейний характер (bang-bang control). У силу цього характеру, для знаходження траєкторій руху ми отримуємо різні постановки задач для різних етапів польоту апарату:

1. **Фаза вільного падіння** ($u(t) = 0$ на $t \in [0, t_s]$):

$$\begin{cases} \dot{h}(t) = v(t), \\ \dot{v}(t) = -g, \\ \dot{m}(t) = 0. \end{cases} \quad (4.1)$$

У фазі падіння диференціальна система має лише початкові (ліві крайові) умови: $h(0) = h_0, v(0) = v_0, m(0) = m_0$. Найбільш раціональним способом знаходження траєкторії апарату для цієї фази є аналітичне розв'язання задачі Коші. Рівняння для цієї траєкторії були знайдені в другому розділі і представляють собою:

$$\begin{cases} h_{free}(t) = h_0 + v_0 t - \frac{gt^2}{2}, \\ v_{free}(t) = v_0 - gt, \\ m_{free}(t) = m_0. \end{cases} \quad (4.2)$$

Таким чином, стан системи в будь-який момент часу $t \leq t_s$ визначається однозначно і не потребує чисельного інтегрування.

2. **Фаза гальмування** ($u(t) = u_{max}$ на $t \in (t_s, \tau]$):

$$\begin{cases} \dot{h}(t) = v(t), \\ \dot{v}(t) = \frac{u_{max}}{m(t)} - g, \\ \dot{m}(t) = -u_{max}k. \end{cases} \quad (4.3)$$

Складність цього етапу зумовлена тим, що ми маємо не лише умови на правому кінці: $h(\tau) = 0, v(\tau) = 0$, але й умови на лівому кінці: $h(t_s) = h_{free}(t_s), v(t_s) =$

$v_{free}(t_s), m(t_s) = m_{free}(t_s)$, які зумовлені станом апарата в кінці першого етапу. Головна проблема полягає в тому, що час перемикавання t_s є невідомим параметром, який безпосередньо впливає на виконання термінальних умов.

Відповідно, задача знаходження траєкторії руху зводиться не до простого розв'язання задачі Коші, а до пошуку невідомого часу перемикавання t_s за допомогою чисельних методів. Тому в подальшому викладі ми розглянемо чисельний алгоритм розв'язання цієї крайової задачі та його програмну реалізацію, що дозволить нам отримати фінальні графіки траєкторій нашого космічного апарата.

4.2 Чисельний метод розв'язання крайової задачі: Метод пристрілки

Проблема знаходження траєкторій для систем із заданими крайовими умовами є класичною в обчислювальній математиці. Згідно з роботою [10, §18], одним із найбільш ефективних і поширених способів її розв'язання є **метод пристрілки** (shooting method). Його суть полягає у зведенні крайової задачі до послідовності задач Коші шляхом введення невідомого параметра — параметра «пристрілки» — та мінімізації функції нев'язки на правому кінці часового інтервалу.

У класичних задачах, які описані в джерелі [10, §18], параметром пристрілки зазвичай виступає відсутня початкова умова (наприклад, початкова швидкість або кут). У нашому ж випадку роль такого параметра відіграє час ввімкнення гальмівного двигуна t_s .

Позначимо фазовий стан апарата як вектор $X(t) = [h(t), v(t), m(t)]^T$. На кожній ітерації алгоритму для обраного (тестового) значення t_s формується початкова умова для фази гальмування. Вона обчислюється аналітично з рівнянь вільного падіння (4.2):

$$X(t_s) = X_{free}(t_s) = \begin{bmatrix} h_0 + v_0 t_s - \frac{gt_s^2}{2} \\ v_0 - gt_s \\ m_0 \end{bmatrix}.$$

Отриманий вектор $X(t_s)$ стає початковою умовою для задачі Коші на активній фазі польоту ($t > t_s$), де чисельне інтегрування системи (4.3):

$$\begin{cases} \dot{h}(t) = v(t), \\ \dot{v}(t) = \frac{u_{max}}{m(t)} - g, \\ \dot{m}(t) = -u_{max}k \end{cases}, \quad X(t_s) = X_{free}(t_s),$$

триває до невідомого моменту часу τ , який визначається виконанням першої термінальної умови — обнуленням швидкості:

$$v(\tau; t_s) = 0.$$

Функція $v(\tau; t_s)$ виражає значення кінцевої швидкості апарата в залежності від t_s .

Після зупинки інтегрування перевіряється друга термінальна умова — висота апарата над поверхнею. Значення цієї висоти утворює **функцію нев'язки** $F(t_s)$:

$$F(t_s) = h(\tau; t_s).$$

Згідно з методом стрільби, точний (оптимальний) час перемикання t_s^* є коренем нелінійного рівняння:

$$F(t_s^*) = 0. \quad (4.4)$$

Таким чином, складна крайова задача оптимізації траєкторії трансформується в алгоритмічно прозору задачу пошуку кореня скалярної функції (4.4).

Програмна реалізація безпосередньо етапу «пострілу» (чисельного інтегрування активної фази та локалізації термінального стану) наведена нижче. Дана функція `shoot` обчислює фазову траєкторію для заданого часу перемикання t_s (змінна `t0`) і застосовує алгоритм лінійної інтерполяції (параметр θ) для точної фіксації моменту обнулення швидкості, усуваючи похибку дискретизації методу Рунге-Кутта.

```
def shoot(L0, t0, vtau, args, dt=1):

    start_cond = free_lunar_system(t0, L0, args[1])
    L = [start_cond]
    T = [t0]

    iteration = 0

    while True:

        iteration += 1

        m_prev, v_prev, h_prev = L[-1]
        t_prev = T[-1]

        # Крок чисельного інтегрування
        m_curr, v_curr, h_curr = num_sol_diff_system(lunar_system, [m_prev, v_prev,
```

```

h_prev], dt, args)

# Перевірка виконання термінальної умови (обнулення швидкості)
if v_curr*v_prev <= 0:

    dv = v_curr - v_prev
    if abs(dv) > 1e-9:
        theta = (vtau - v_prev) / dv
    else:
        theta = 0

    t_final = t_prev + theta * dt

    # Лінійна інтерполяція фінального стану
    h_final = h_prev + theta * (h_curr - h_prev)
    v_final = vtau
    m_final = m_prev + theta * (m_curr - m_prev)

    L.append([m_final, v_final, h_final])
    T.append(t_final)
    return np.array(L), T

# Умова зупинки при зіткненні (промах)
elif h_curr <= -L0[2]:
    return None, None

L.append([m_curr, v_curr, h_curr])
T.append(T[-1]+dt)

```

Процес ітеративного використання цієї функції для пошуку оптимального часу t_s^* (наближення $F(t_s)$ до нуля) за допомогою методів інкрементального пошуку та половинного ділення детально розглянутий у наступному підрозділі.

4.3 Аналіз функції нев'язки та алгоритм локалізації кореня

Вибір правильної функції нев'язки є критично важливим етапом методу стрільби, оскільки від її математичних властивостей залежить збіжність чисельних методів. Для

крайової задачі м'якої посадки існують два альтернативні підходи до формування функції нев'язки:

1. Зупиняти інтегрування при досягненні нульової висоти ($h = 0$) і перевіряти залишкову швидкість $v(\tau; t_s)$.
2. Зупиняти інтегрування при досягненні нульової швидкості ($v = 0$) і перевіряти залишкову висоту $h(\tau; t_s)$.

Для обґрунтування вибору був проведений графічний аналіз залежності обох потенційних нев'язок від часу перемикання t_s (див. рис. 1).

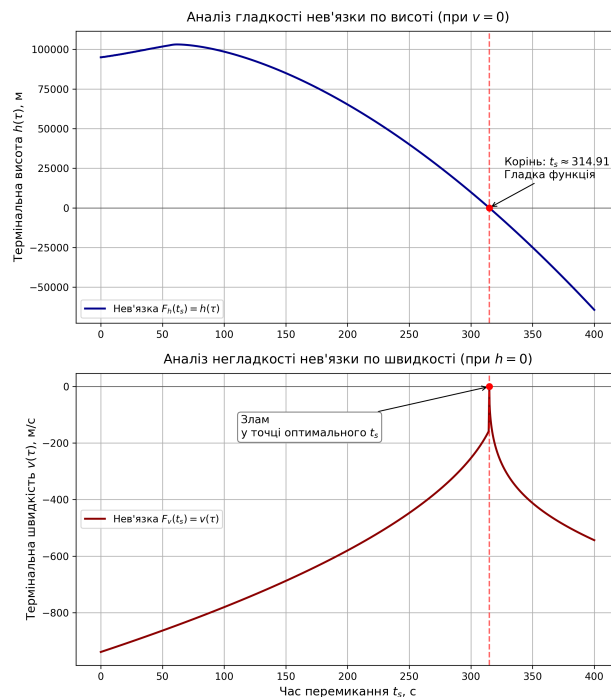


Рис. 1: Графіки нев'язок

Графічне зображення нев'язок гарно відображає, що функція швидкості має яскраво виражений злам у точці оптимального перемикання. Відсутність гладкості робить неможливим застосування ефективних градієнтних методів, таких як метод Ньютона-Рафсона (який вимагає обчислення похідної $F'(t_s)$) або класичного методу січних, оскільки вони втрачають стійкість або розбігаються поблизу зламу [10, §9.1].

Натомість графік функції кінцевої висоти $h(\tau; t_s)$ за умови $v = 0$ є гладкою, монотонною кривою. Саме тому як функцію нев'язки $F(t_s)$ було обрано термінальну висоту:

$$F(t_s) = h(\tau; t_s) \quad \text{при} \quad v(\tau) = 0.$$

Відповідно до рекомендацій з обчислювальної математики [10, §9.1], надійний пошук кореня нелінійного рівняння складається з двох етапів: спочатку корінь необхідно локалі-

зувати (bracket the root) — знайти інтервал, на кінцях якого функція змінює знак, а потім уточнити його розташування.

Оскільки апіорні межі часу перемикання невідомі, був розроблений гібридний алгоритм послідовного сканування з адаптивним дробленням кроку (модифікований метод бісекції). Відмінною рисою алгоритму є використання критерію зміни знака цільової функції, що забезпечує стійкість пошуку незалежно від складності балістичної траєкторії.

Суть розробленого алгоритму (реалізованого у функції `strike_methods`) полягає в наступному:

- Алгоритм починає рух від початкового наближення $t_1 = 0$ та наступної пробної точки $t_2 = \Delta t$ із фіксованим кроком.
- На кожному ітераційному кроці розв’язується задача Коші (виклик функції `shoot`) та обчислюються значення цільової нев’язки (висоти зависання) на кінцях поточного інтервалу: F_1 та F_2 .
- Перевіряється умова локалізації кореня: якщо $F_1 \cdot F_2 \geq 0$, функція не змінила знак. Це означає, що точка оптимального ввімкнення знаходиться далі. Алгоритм зсуває вікно пошуку вперед ($t_1 = t_2$, $t_2 = t_1 + \Delta t$).
- Якщо $F_1 \cdot F_2 < 0$, відбувся «переліт» (функція перетнула нуль). Це свідчить про те, що корінь локалізовано всередині поточного відрізка. Алгоритм зменшує крок пошуку вдвічі ($\Delta t := \Delta t/2$) і зміщує праву межу t_2 назад, звужуючи інтервал.
- Ітерації тривають доти, доки абсолютне значення нев’язки $|F_2|$ не стане меншим за суму цільової висоти та заданої допустимої похибки ϵ .

Програмна реалізація даного алгоритму наведена нижче:

```
def strike_methods(L0, Ltau, args, dt=10, eps=0.01):  
  
    t1, t2 = 0, dt  
  
    L, T = shoot(L0, t1, Ltau[0], args)  
    F1 = L[-1, 2]  
  
    L, T = shoot(L0, t2, Ltau[0], args)  
    F2 = L[-1, 2]
```

```

while abs(F2) >= Ltau[1] + eps:

    if F1 * F2 < 0:
        dt *= 0.5
        t2 = t1 + dt
    else:
        t1 = t2
        F1 = F2
        t2 = t1 + dt

L, T = shoot(L0, t2, Ltau[0] + eps, args)
F2 = L[-1, 2]

return L, T

```

Такий підхід не потребує обчислення похідних, є стійким до похибок округлення і гарантує збіжність до точного розв’язку завдяки властивості безперервності цільової функції (4.4).

4.4 Чисельне розв’язання систем диференціальних рівнянь методом Рунге-Кутта 4-го порядку

Для отримання фазових траєкторій системи диференціальних рівнянь активної фази гальмування на кожному ітераційному етапі методу стрільби було застосовано класичний метод Рунге-Кутта четвертого порядку точності. Згідно з роботою [11, §5.1], цей метод забезпечує високий баланс між обчислювальною стійкістю та точністю, що є критично важливим для жорстких умов моделювання динаміки космічного апарата. Оскільки права частина досліджуваної диференціальної системи явно не залежить від часу, ми розглядаємо задачу Коші у векторному вигляді:

$$\dot{Y} = f(Y), \quad Y(t_s) = Y_0, \quad (4.5)$$

де $Y = [m, v, h]^T$ — фазовий вектор стану апарата. Значення фазового вектора на наступному часовому кроці $t_{n+1} = t_n + \Delta t$ обчислюється синхронно для всіх змінних за схемою:

$$Y_{n+1} = Y_n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4), \quad (4.6)$$

де векторні коефіцієнти методу (нахили на різних етапах кроку) визначаються як:

$$\begin{cases} k_1 = f(Y_n), \\ k_2 = f\left(Y_n + \frac{\Delta t}{2} k_1\right), \\ k_3 = f\left(Y_n + \frac{\Delta t}{2} k_2\right), \\ k_4 = f(Y_n + \Delta t k_3). \end{cases}$$

Глобальна похибка даного методу складає $O(\Delta t^4)$, що дозволило досягти необхідної точності моделювання швидкості та висоти при прийнятних витратах часу на обчислення. Програмна реалізація векторного методу Рунге-Кутта та функції правих частин диференціальної системи наведена нижче:

```
def lunar_system(Y, args):

    """
    Векторна функція правих частин автономної системи. Y = [m, v, h] args = [u, g, k, m]
    """

    m, v, _ = Y
    u, g, k, min_m = args

    dh_dt = v

    # Перевірка наявності палива для генерації тяги
    if m >= min_m:
        dv_dt = (u / m) - g
        dm_dt = -u * k
    else:
        dv_dt = -g
        dm_dt = 0

    return np.array([dm_dt, dv_dt, dh_dt])

def num_sol_diff_system(F, Y0, dt, args):

    """
    Синхронний крок інтегрування методом Рунге-Кутта 4-го порядку для вектор-функції с
```

```
"""
```

```
k_1 = F(Y0, args)
k_2 = F(Y0 + (dt / 2) * k_1, args)
k_3 = F(Y0 + (dt / 2) * k_2, args)
k_4 = F(Y0 + dt * k_3, args)
Y_next = Y0 + dt * (k_1 + 2*k_2 + 2*k_3 + k_4) / 6

return Y_next
```

5 Результати обчислювальних експериментів

Для перевірки адекватності розробленої програми було проведено серію експериментів. Тестування відбувалося на трьох різних сценаріях, що моделюють реальні та екстремальні умови космічних місій. У всіх експериментах гравітаційне прискорення Місяця приймалося як $g = 1.622 \text{ м/с}^2$.

5.1 Експеримент 1: Верифікація програми

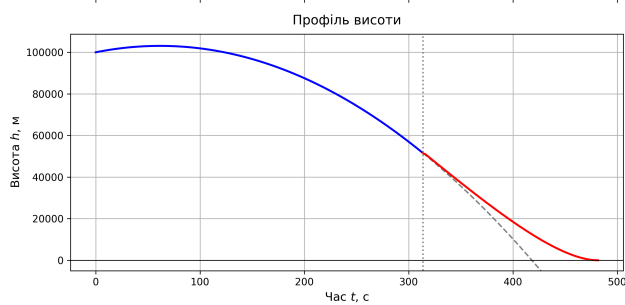
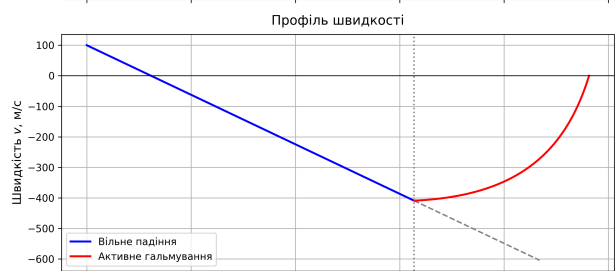
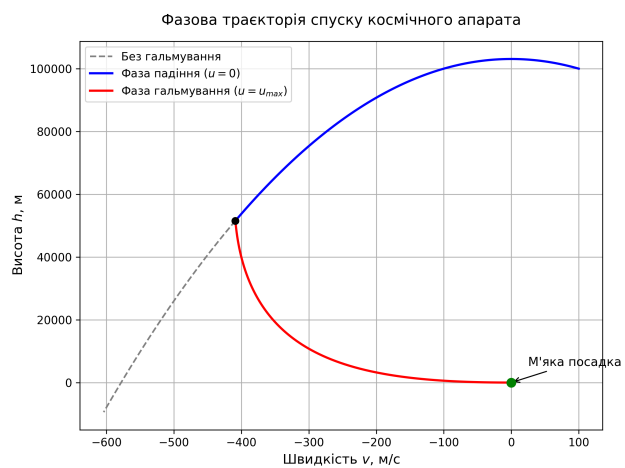
Перший сценарій базується на параметрах тестової задачі з джерела [12], що дозволяє верифікувати точність нашого методу шляхом порівняння з відомими результатами.

Початкові умови та параметри системи:

- Початкова висота: $h_0 = 100\,000 \text{ м}$
- Початкова швидкість: $v_0 = 100 \text{ м/с}$
- Маса апарата: $m_0 = 224 \text{ кг}$ (з них паливо $m_{fuel} = 204 \text{ кг}$)
- Максимальна тяга: $u = 400 \text{ Н}$
- Коефіцієнт витрати палива: $k = 2.883 \cdot 10^{-3}$

Результати моделювання: В дужках приведені результати обчислень тестової задачі з джерела [12].

Параметр	Фаза перемикання (t_s)	Термінальна фаза (τ)
Час, с	$t_s = 313.75(312.85)$	$\tau = 481.61(481.84)$
Висота, м	$h_{t_s} = 51540.28(51904.47)$	$h_\tau = -0.0008$
Швидкість, м/с	$v_{t_s} = -408.91(-407.45)$	$v_\tau = 0.01$
Маса, кг	$m_{t_s} = 224.0$	$m_\tau = 31.58(32, 47)$



Результати демонструють високий рівень збігу з контрольним прикладом.

5.2 Експеримент 2: Посадка важкого модуля (з нульовою початковою швидкістю)

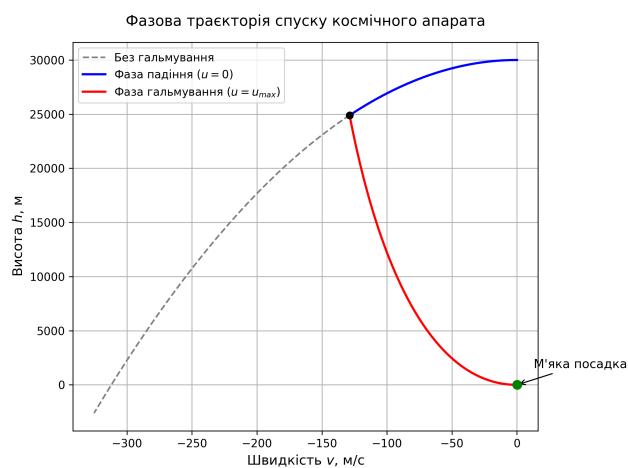
Другий сценарій моделює посадку масивного апарата (аналог сучасних місячних модулів), який починає вільне падіння зі стану спокою з відносно низької орбіти.

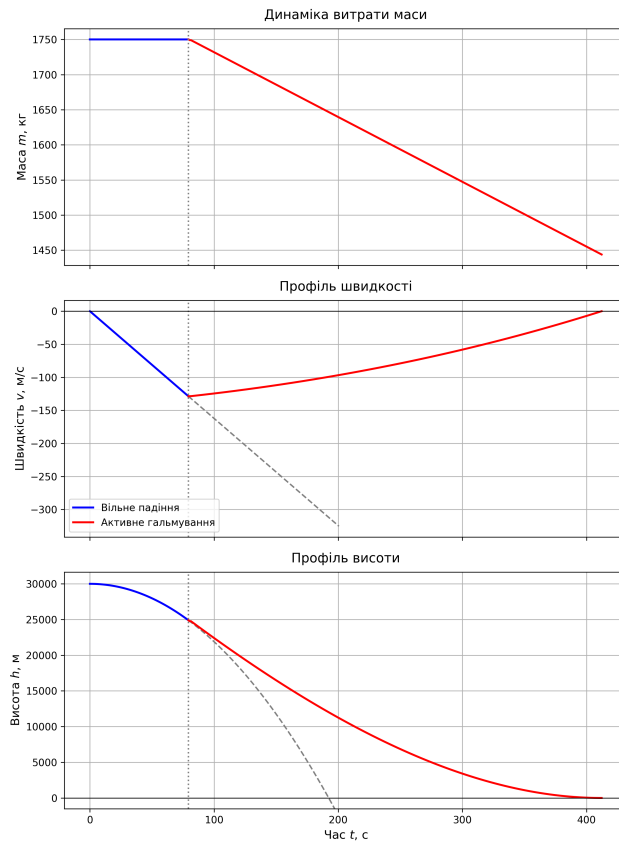
Початкові умови та параметри системи:

- Початкова висота: $h_0 = 30\,000$ м
- Початкова швидкість: $v_0 = 0$ м/с
- Маса апарата: $m_0 = 1750$ кг (з них паливо $m_{fuel} = 1100$ кг)
- Сумарна тяга: $u = 3200$ Н
- Коефіцієнт витрати палива: $k = 2.883 \cdot 10^{-4}$

Результати моделювання:

Параметр	Фаза перемикання (t_s)	Термінальна фаза (τ)
Час, с	$t_s = 79.35$	$\tau = 412.13$
Висота, м	$h_{ts} = 24893.02$	$h_\tau = 0.0057$
Швидкість, м/с	$v_{ts} = -128.71$	$v_\tau = 0.01$
Маса, кг	$m_{ts} = 1750.0$	$m_\tau = 1443.91$





Незважаючи на велику масу та інерцію, алгоритм успішно знайшов точку перемикання. Витрата палива склала ≈ 306 кг, що є оптимальним показником для заданих умов.

5.3 Експеримент 3: Стрес-тест зі складною кінематикою (балістичний стрибок)

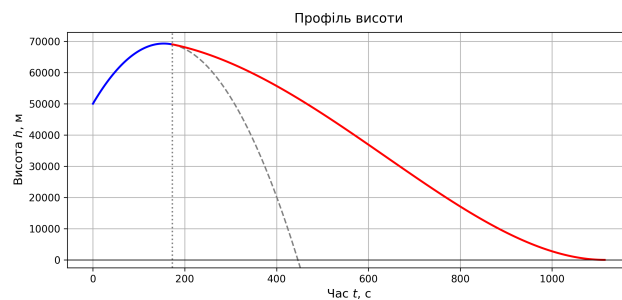
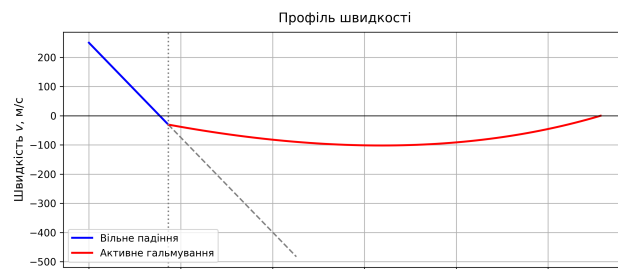
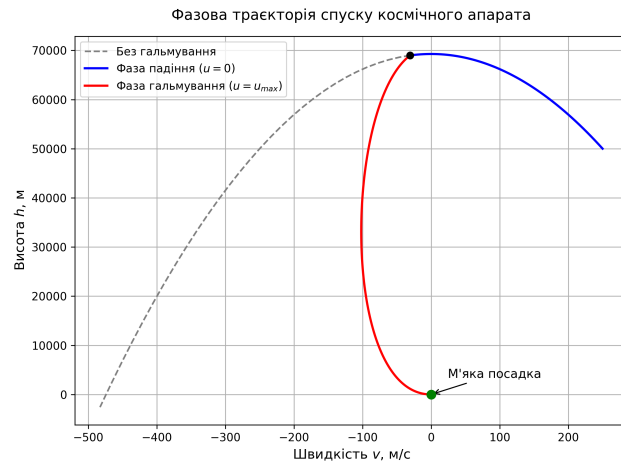
Третій сценарій є найскладнішим обчислювальним випадком. Апарат має початкову позитивну швидкість $v_0 = 250$ м/с. Це означає, що до початку падіння він здійснює балістичний набір висоти.

Початкові умови та параметри системи:

- Початкова висота: $h_0 = 50\,000$ м
- Початкова швидкість: $v_0 = 250$ м/с (висхідна)
- Маса апарата: $m_0 = 450$ кг (з них паливо $m_{fuel} = 300$ кг)
- Тяга: $u = 600$ Н
- Коефіцієнт витрати палива: $k = 2.883 \cdot 10^{-4}$

Результати моделювання:

Параметр	Фаза перемикання (t_s)	Термінальна фаза (τ)
Час, с	$t_s = 173.08$	$\tau = 1114.68$
Висота, м	$h_{ts} = 68975.01$	$h_\tau = -0.0560$
Швидкість, м/с	$v_{ts} = -30.74$	$v_\tau = 0.10$
Маса, кг	$m_{ts} = 450.0$	$m_\tau = 287.30$



Алгоритм коректно відпрацював зміну знака швидкості: апарат спочатку досяг пікової висоти (близько 69 км), після чого алгоритм розрахував ідеальну точку ввімкнення

двигуна. Термінальні умови виконані з точністю $v = 0.1$ м/с, що підтверджує абсолютну обчислювальну стійкість створеного математичного забезпечення до нестандартних початкових умов.